



Amrita Vishwa Vidyapeetham, Chennai Campus

Department of Computer Science and Engineering

23CSE463 – Quantum Computing

Work Sheet 2

NAME : Ganath Avinash G R

ROLL NUMBER : CH.SC.U4CSE24118

CLASS : CSE-B

ACADEMIC YEAR : 2025 - 26

Date : 22/07/2026



WORKSHEET- 2

EX.NO : 2

DATE : 18/06/2026

TITLE : STUDY AND VISUALIZATION OF SINGLE-QUBIT STATES USING THE BLOCH SPHERE

AIM:

To study the representation of single-qubit quantum states using the Bloch Sphere and visualize the effect of various quantum gates on qubit states using the Qiskit framework. The objective is to understand quantum state representation, superposition, and state transformations through Bloch Sphere visualization.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit Quantum Computing Framework
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

Introduction to the Bloch Sphere:

The Bloch Sphere is a geometrical representation of the state space of a single qubit. It provides an intuitive way to visualize and analyze quantum states and their transformations. Every pure state of a single qubit can be represented as a point on the surface of the Bloch Sphere.

Single-Qubit Quantum States:

Unlike a classical bit, which can exist only in the states 0 or 1, a qubit can exist in a superposition of both states. The general state of a qubit is represented as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$



where α and β are complex probability amplitudes satisfying:

$$|\alpha|^2 + |\beta|^2 = 1$$

Bloch Sphere Representation:

A single-qubit state can be represented on the Bloch Sphere using two parameters, θ and ϕ , as:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{(i\phi)\sin\left(\frac{\theta}{2}\right)}|1\rangle$$

where:

- θ represents the polar angle.
- ϕ represents the azimuthal angle.

These angles determine the position of the qubit on the Bloch Sphere.

Computational Basis States:

The computational basis states are represented by the poles of the Bloch Sphere:

- North Pole $\rightarrow |0\rangle$
- South Pole $\rightarrow |1\rangle$

These states correspond to the classical binary states.

Superposition States:

States located on the equator of the Bloch Sphere represent equal superpositions of $|0\rangle$ and $|1\rangle$. The Hadamard gate is commonly used to generate such superposition states.

Quantum Gates and State Rotation:

Quantum gates can be interpreted as rotations of the qubit state vector on the Bloch Sphere.

- Pauli-X Gate: Rotation about the X-axis.
- Pauli-Y Gate: Rotation about the Y-axis.
- Pauli-Z Gate: Rotation about the Z-axis.
- Hadamard Gate: Creates superposition states.

These gates change the position of the qubit state on the Bloch Sphere.

Bloch Sphere Visualization Using Qiskit:



Qiskit provides visualization tools to represent quantum states on the Bloch Sphere. By applying different quantum gates and generating state vectors, the resulting qubit states can be visualized and analyzed effectively.

Significance of the Bloch Sphere:

The Bloch Sphere is widely used for understanding quantum state evolution, quantum gate operations, superposition, and quantum algorithms. It serves as a fundamental visualization tool in quantum computing and quantum information processing.

PROCEDURE:

1. Open Jupyter Notebook or Visual Studio Code.
2. Import the required Qiskit libraries and visualization modules.
3. Create a single-qubit quantum circuit.
4. Initialize the qubit in the computational basis state $|0\rangle$.
5. Generate the state vector of the qubit.
6. Visualize the quantum state using the Bloch Sphere.
7. Apply quantum gates such as X, Y, Z, and H to the qubit.
8. Observe the changes in the qubit state on the Bloch Sphere.
9. Compare the Bloch Sphere representations obtained for different quantum gates.
10. Analyze and verify the resulting quantum states.



PROGRAM:

Program 1: Create a quantum circuit with a single qubit in the $|0\rangle$ state and visualize it on the Bloch Sphere.

CODE:

```
✓ from qiskit import QuantumCircuit
  from qiskit.visualization import plot_bloch_vector
  import matplotlib.pyplot as plt

  # Create a 1-qubit circuit
  qc = QuantumCircuit(1)

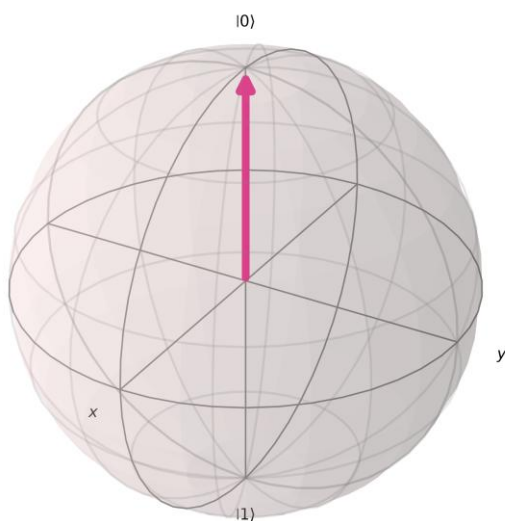
  # Display circuit
  print(qc)

  # Create Bloch Sphere for  $|0\rangle$ 
  fig = plot_bloch_vector([0, 0, 1])

  # Save the Bloch Sphere
  fig.savefig("bloch_sphere.png", dpi=300, bbox_inches="tight")

  # Show it
  plt.show()
✓ 0.3s
```

OUTPUT:





INFERENCE:

The qubit is initially in the $|0\rangle$ computational basis state. On the Bloch Sphere, it appears at the north pole, corresponding to coordinates:

$$(x, y, z) = (0, 0, 1)$$

Measurement always produces 0.

Program 2: Apply the Pauli-X gate to a single qubit and visualize the resulting $|1\rangle$ state on the Bloch Sphere.

CODE:

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create a 1-qubit circuit
qc = QuantumCircuit(1)

# Apply Pauli-X gate
qc.x(0)

# Display the circuit
print(qc)

#  $|1\rangle$  corresponds to Bloch vector (0, 0, -1)
fig = plot_bloch_vector([0, 0, -1])

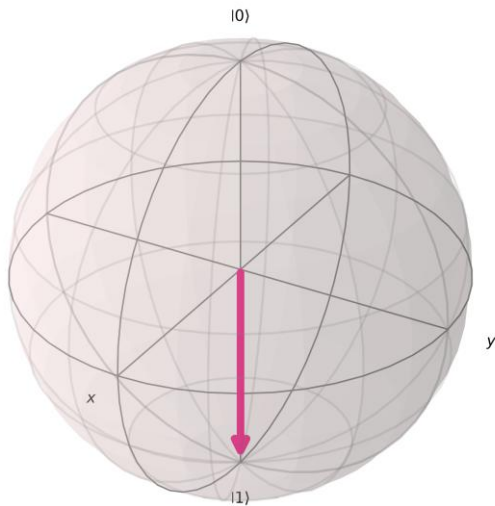
# Save the Bloch Sphere as a clear image
fig.savefig(
    "pauli_x_bloch_sphere.png",
    dpi=300,
    bbox_inches="tight"
)

print("Bloch Sphere saved as pauli_x_bloch_sphere.png")
```

OUTPUT:

```
q: ──[ X ]──
```

```
Bloch Sphere saved as pauli_x_bloch_sphere.pr
```



INFERENCE:

The Pauli-X gate flips the qubit from $|0\rangle$ to $|1\rangle$. The Bloch vector moves from the north pole to the south pole.

Program 3: Create a quantum circuit using the Hadamard gate and visualize the superposition state on the Bloch Sphere.

CODE:

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create a 1-qubit circuit
qc = QuantumCircuit(1)

# Apply Hadamard gate
qc.h(0)

# Display the circuit
print(qc)

# Hadamard creates the  $|+\rangle$  superposition state
fig = plot_bloch_vector([1, 0, 0])

# Save the Bloch Sphere
fig.savefig(
    "hadamard_bloch_sphere.png",
    dpi=300,
    bbox_inches="tight"
)

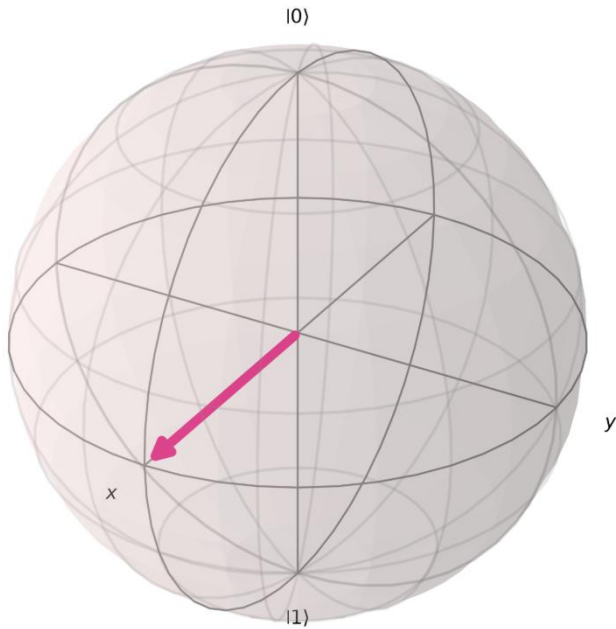
print("Bloch Sphere saved as hadamard_bloch_sphere.png")
```



OUTPUT:

```
q: ┌ H ┐
```

Bloch Sphere saved as hadamard_bloch_sphere.png



INFERENCE:

The Hadamard gate converts $|0\rangle$ into an equal superposition of $|0\rangle$ and $|1\rangle$. The Bloch vector moves from the north pole to the positive X-axis on the equator.



Program 4: Apply the Pauli-Y gate to a single qubit and observe the state transformation using the Bloch Sphere.

CODE:

```
# Q4
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create a 1-qubit circuit
qc = QuantumCircuit(1)

# Apply Pauli-Y gate
qc.y(0)

# Display the circuit
print(qc)

# After  $Y|0\rangle = i|1\rangle$ 
# The global phase  $i$  does not affect the Bloch Sphere.
# Therefore, the state is  $|1\rangle$ , with Bloch vector (0, 0, -1)
fig = plot_bloch_vector([0, 0, -1])

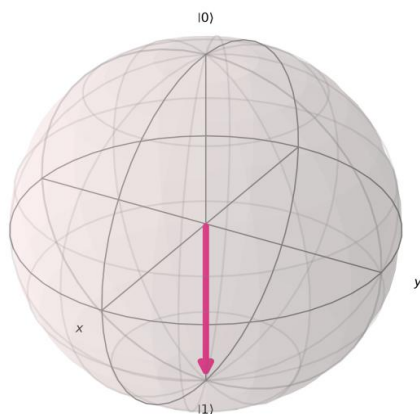
# Save the Bloch Sphere
fig.savefig(
    "pauli_y_bloch_sphere.png",
    dpi=300,
    bbox_inches="tight"
)

print("Bloch Sphere saved as pauli_y_bloch_sphere.png")
```

OUTPUT:

q: $\boxed{Y}$

Bloch Sphere saved as pauli_y_bloch_sphere.png



**INFERENCE:**

The Pauli-Y gate rotates the qubit about the Y-axis. When applied to $|0\rangle$, the resulting state is $i|1\rangle$, which has the same Bloch Sphere position as $|1\rangle$.

Program 5: Apply the Pauli-Z gate to a single qubit and visualize the resulting state on the Bloch Sphere.

CODE:

```
#Q5
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create a 1-qubit circuit
qc = QuantumCircuit(1)

# Apply Pauli-Z gate
qc.z(0)

# Display the circuit
print(qc)

#  $Z|0\rangle = |0\rangle$ 
# Bloch vector for  $|0\rangle = (0, 0, 1)$ 
fig = plot_bloch_vector([0, 0, 1])

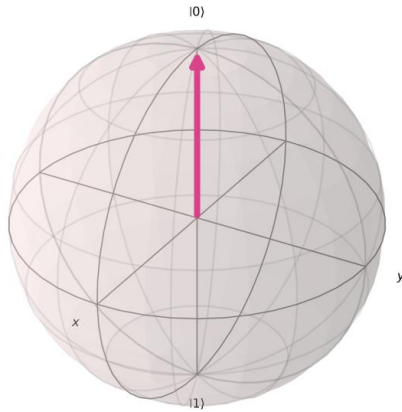
# Save the Bloch Sphere
fig.savefig(
    "pauli_z_bloch_sphere.png",
    dpi=300,
    bbox_inches="tight"
)

print("Bloch Sphere saved as pauli_z_bloch_sphere.png")
```

OUTPUT:

```
q: ┌───┐
    │ Z │
    └───┘

Bloch Sphere saved as pauli_z_bloch_sphere.png
```



INFERENCE:

The Pauli-Z gate introduces a phase change between $|0\rangle$ and $|1\rangle$. Since the initial state is $|0\rangle$, its Bloch Sphere position does not change and it remains at the north pole.

Program 6: Create and visualize the Bloch Sphere representation of the $|+\rangle$ state using the Hadamard gate.

CODE:

```
# Q6
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create a 1-qubit circuit
qc = QuantumCircuit(1)

# Apply Hadamard gate
qc.h(0)

# Display the circuit
print(qc)

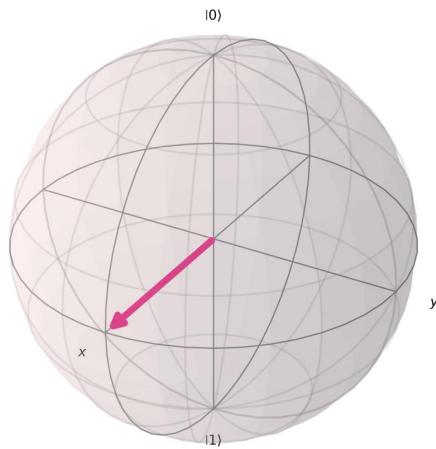
# |+> state has Bloch vector (1, 0, 0)
fig = plot_bloch_vector([1, 0, 0])

# Save the Bloch Sphere
fig.savefig(
    "plus_state_bloch_sphere.png",
    dpi=300,
    bbox_inches="tight"
)

print("Bloch Sphere saved as plus_state_bloch_sphere.png")
```



OUTPUT:



INFERENCE:

The $|+\rangle$ state lies on the positive X-axis of the Bloch Sphere. It is an equal superposition of $|0\rangle$ and $|1\rangle$.

Program 7: Create and visualize the Bloch Sphere representation of the $|-\rangle$ state using the Hadamard and Pauli-Z gates.

CODE:

```
# Q7
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create 1-qubit circuit
qc = QuantumCircuit(1)

# Apply Hadamard gate
qc.h(0)

# Apply Pauli-Z gate
qc.z(0)

# Display circuit
print(qc)

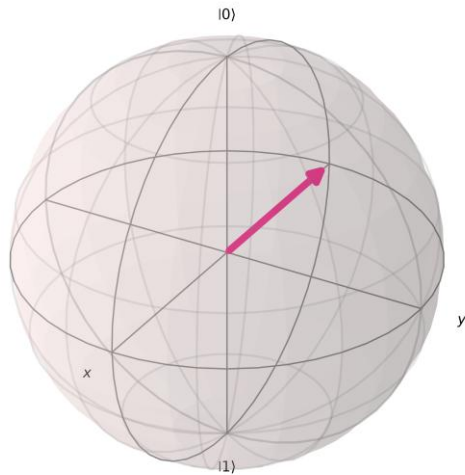
# Bloch vector for  $|-\rangle$  state
#  $|-\rangle$  lies on the -X axis
fig = plot_bloch_vector([-1, 0, 0])

# Save Bloch Sphere
fig.savefig("minus_state_bloch_sphere.png",
            dpi=300,
            bbox_inches="tight")

print("Bloch Sphere saved as minus_state_bloch_sphere.png")
```



OUTPUT:



INFERENCE:

The Pauli-Z gate changes the relative phase between $|0\rangle$ and $|1\rangle$. The Bloch vector moves from the positive X-axis to the negative X-axis, producing the $|-\rangle$ state.

Program 8: Apply a sequence of quantum gates (H followed by X) and observe the resulting state on the Bloch Sphere.

CODE:

```
# Q8
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Create 1-qubit circuit
qc = QuantumCircuit(1)

# Apply H followed by X
qc.h(0)
qc.x(0)

# Display circuit
print(qc)

# Bloch vector for |+> state
fig = plot_bloch_vector([1, 0, 0])

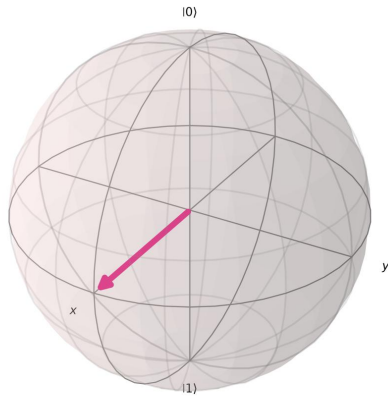
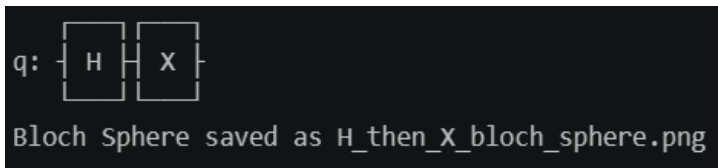
# Save Bloch Sphere
fig.savefig("H_then_X_bloch_sphere.png",
            dpi=300,
            bbox_inches="tight")

print("Bloch Sphere saved as H_then_X_bloch_sphere.png")
```

✓ 0.1s



OUTPUT:



INFERENCE:

The Hadamard gate creates $|+\rangle$. Applying X to $|+\rangle$ leaves it unchanged because $|+\rangle$ is an eigenstate of the X gate with eigenvalue $+1$. Therefore, the final Bloch Sphere position remains on the positive X -axis.



Program 9: Compare the Bloch Sphere representations obtained using Pauli-X, Pauli-Y, and Pauli-Z gates.

CODE:

```
# Q9
from qiskit import QuantumCircuit
from qiskit.visualization import plot_bloch_vector

# Pauli-X
qc_x = QuantumCircuit(1)
qc_x.x(0)
print("Pauli-X:")
print(qc_x)

fig_x = plot_bloch_vector([0, 0, -1])
fig_x.savefig("pauli_x_bloch.png", dpi=300, bbox_inches="tight")

# Pauli-Y
qc_y = QuantumCircuit(1)
qc_y.y(0)
print("Pauli-Y:")
print(qc_y)

fig_y = plot_bloch_vector([0, 0, -1])
fig_y.savefig("pauli_y_bloch.png", dpi=300, bbox_inches="tight")

# Pauli-Z
qc_z = QuantumCircuit(1)
qc_z.z(0)
print("Pauli-Z:")
print(qc_z)

fig_z = plot_bloch_vector([0, 0, 1])
fig_z.savefig("pauli_z_bloch.png", dpi=300, bbox_inches="tight")

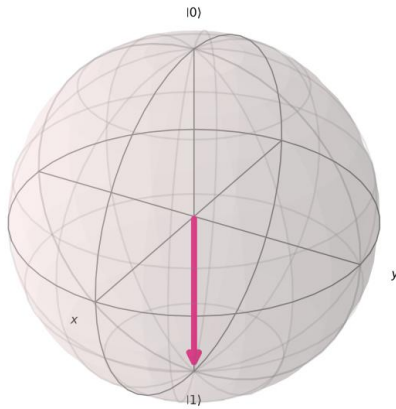
print("All Bloch Sphere images saved.")
```

✓ 0.6s

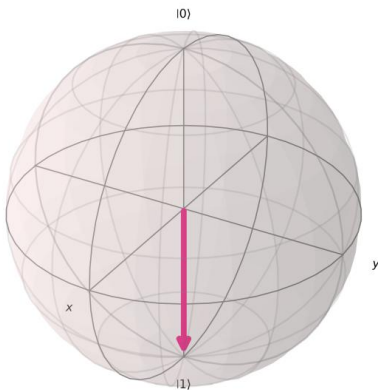


OUTPUT:

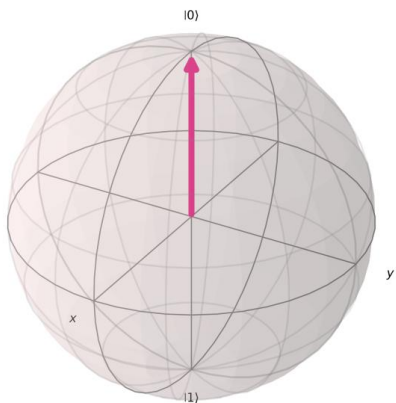
X-Gate

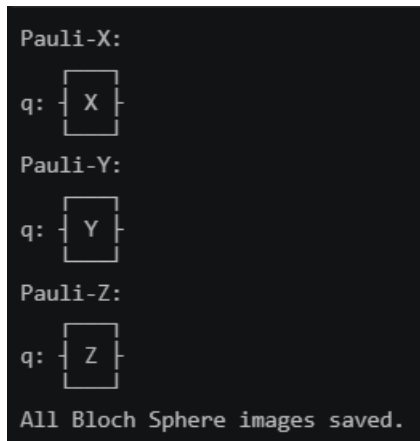


Y-Gate



Z-Gate





INFERENCE:

The Pauli-X and Pauli-Y gates both transform $|0\rangle$ to states having the same measurement probabilities as $|1\rangle$, so they appear at the south pole. The difference is the phase introduced by Y.

The Z gate changes the phase of $|1\rangle$, but since the input is $|0\rangle$, the state remains visually at the north pole.



Program 10: Generate the state vector of a single-qubit quantum circuit and visualize its corresponding Bloch Sphere representation using Qiskit.

CODE:

```
# Q10
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_vector

# Create a single-qubit circuit
qc = QuantumCircuit(1)

# Apply Hadamard gate
qc.h(0)

# Display the circuit
print("Quantum Circuit:")
print(qc)

# Generate the state vector
state = Statevector.from_instruction(qc)

print("\nState Vector:")
print(state)

# Bloch vector for |+> state
fig = plot_bloch_vector([1, 0, 0])

# Save Bloch Sphere
fig.savefig("statevector_bloch_sphere.png",
            dpi=300,
            bbox_inches="tight")

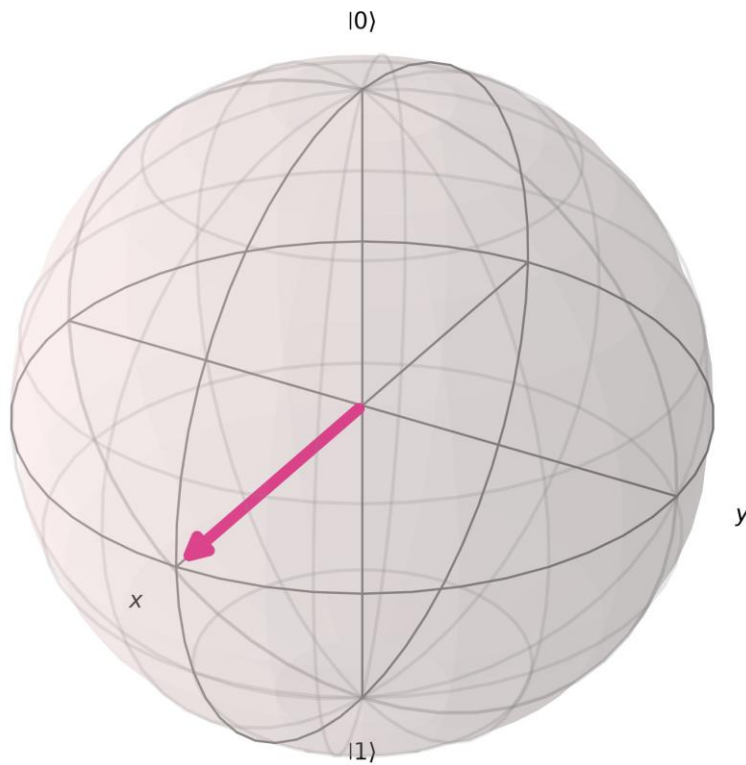
print("\nBloch Sphere saved as statevector_bloch_sphere.png")
```

OUTPUT:

```
Quantum Circuit:
q: ──[ H ]──

State Vector:
Statevector([0.70710678+0.j, 0.70710678+0.j],
            dims=(2,))

Bloch Sphere saved as statevector_bloch_sphere.png
```



INFERENCE:

The state vector gives the complete mathematical description of the single qubit, while the Bloch Sphere gives a geometric representation of the same state.

The R_Y and R_Z gates rotate the state vector to a new point on the Bloch Sphere. The measurement probabilities are determined by the amplitudes of the resulting state vector.



VIVA Q&A:

1. What is a Bloch Sphere?

A Bloch Sphere is a geometrical representation of the state space of a single qubit. Every pure single-qubit state can be represented as a point on the surface of the sphere.

2. Why is the Bloch Sphere used in quantum computing?

It is used to intuitively understand quantum state representation, superposition, quantum gate operations, and the evolution of a single-qubit state.

3. How is a single-qubit state represented on the Bloch Sphere?

A single-qubit state is represented using two parameters, θ and ϕ . θ represents the polar angle and ϕ represents the azimuthal angle. These angles determine the position of the qubit on the Bloch Sphere.

4. What is quantum superposition?

Quantum superposition is the ability of a qubit to exist in a combination of both $|0\rangle$ and $|1\rangle$ states. Equal superposition states are located on the equator of the Bloch Sphere.

5. What is the mathematical representation of a qubit?

A qubit is mathematically represented as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex probability amplitudes satisfying $|\alpha|^2 + |\beta|^2 = 1$.

6. What is the effect of the Hadamard gate on a qubit?

The Hadamard gate creates a superposition state. When applied to $|0\rangle$, it produces $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.

7. What is the function of the Pauli-X gate?

The Pauli-X gate flips the qubit state from $|0\rangle$ to $|1\rangle$ and from $|1\rangle$ to $|0\rangle$. It represents a rotation about the X-axis of the Bloch Sphere.

8. What is the difference between the Pauli-X, Pauli-Y, and Pauli-Z gates?

The Pauli-X gate represents rotation about the X-axis, the Pauli-Y gate represents rotation about the Y-axis, and the Pauli-Z gate represents rotation about the Z-axis of the Bloch Sphere.



9. Which Qiskit function is used to visualize a Bloch Sphere?

The Qiskit function `plot_bloch_multivector()` is used to visualize the Bloch Sphere representation of a quantum state.

10. What is the significance of Bloch Sphere representation in quantum computing?

The Bloch Sphere provides an intuitive way to understand quantum state evolution, quantum gate operations, superposition, and quantum algorithms. It helps visualize how quantum gates transform a qubit state.

RESULT:

The single-qubit states were successfully studied and visualized using the Bloch Sphere. The effects of **Pauli-X, Pauli-Y, Pauli-Z, and Hadamard gates** on qubit states were observed using **Qiskit**. The state vectors, measurements, and corresponding **Bloch Sphere** representations were analyzed successfully. Thus, the concepts of quantum superposition, state transformation, and single-qubit quantum gate operations were verified.